Answers of Algorithms and Data Structures, Exam 11 Aug 2025

The questions start on page 1.

It is strongly preferred that you fit your answers on these sheets. If you really must, you can use a separate sheet of paper instead. Please indicate that clearly.

Your name:

Date of Birth (DOB, dd mm yyyy):

Course Code:

Algorithms and Data Structures

Exam 11 Aug 2025

Thore Husfeldt, Riko Jacob, and Holger Dell, ITU

Instructions

What to bring. You can bring any written aid you want. This includes the course book and a dictionary. In fact, these two things are the only aids that make sense, so I recommend you bring them and only them. But if you want to bring other books, notes, print-out of code, old exams, or today's newspaper you can do so. (It won't help.) Electronic devices like mobile phones, pocket calculators, computers or e-book readers are not allowed.

Answering multiple-choice questions. In the multiple-choice questions, there is one and only one correct answer. However, to demonstrate partial knowledge, you are allowed to check 2 or more boxes, but this earns you less than full points for that question.

number of checked boxes	0	1	2	3	4
points if correct answer checked		1	0.5	0.21	0
points if correct answer not checked	0	−0.33	−0.5	−0.62	

In particular, the best thing is to only check the correct answer, and the worst thing is to check all answers but the correct one. If you don't check anything (or check *all* boxes) your score is 0. Also, if you check boxes at random, your expected score is 0. (Just to make sure: a question that is not multiple-choice cannot give you negative points.)

Where to write. Mark your answers on pages 1–**??**. If you really have to, you may use separate sheets of paper for the free text questions instead, but please be clear about it (cross out everything and write "see separate paper, page 1" or something like that.) For the love of all that is Good and Holy, write legibly. Hand in pages 1–**??**, and any separate sheet(s) of paper. Do not hand in pages 1–9, they will not be read.

Exam questions

1. Analysis of algorithms

(a) (*1 pt.*) Which pair of functions satisfy $f(N) \sim g(N)$?

- A $f(N) = 10$ and $g(N) = 2$
- B $f(N) = \log_{10}(10^N)$ and $g(N) = \log_2(2^N)$
- C $f(N) = \log_{10}(N^{10})$ and $g(N) = \log_2(N^2)$
- D $f(N) = \sqrt[2]{N}$ and $g(N) = \sqrt[10]{N}$

(b) (*1 pt.*) Which pair of functions satisfy $f(N) \in O(g(N))$ for $N \geq 2$?

- A $f(N) = \sqrt{N^3 \cdot \log_2 N}$ and $g(N) = N \cdot \log_2 N$
- B $f(N) = 1.5^N$ and $g(N) = N^{1.5}$
- C $f(N) = N^2/(\log_2 N)$ and $g(N) = \sqrt{N^4 \cdot \log_2 N}$
- D $f(N) = (N+1)^2$ and $g(N) = \sqrt{N^2 + 2N + 1}$

(c) (*1 pt.*) How many stars are printed?

```python
# python
i = N
while i > 2:
    i = i // 2
    print("*")
```

```java
// java
for (int i = N; i > 2; i = i/2)
    StdOut.print("*");
```

A $\sim \log N$ B $\sim N$ C $\sim N \log N$ D $\sim \frac{1}{2}N^2$

(d) (*1 pt.*) How many stars are printed?

```python
# python
s = 2
while s < N:
    print("*")
    s = s + 3
print("****")
```

```java
// java
int s = 2;
while (s < N) {
    StdOut.print("*");
    s = s + 3;
}
StdOut.print("****");
```

A $\sim \frac{1}{6} \cdot N$ B $\sim \frac{1}{3} \cdot N$ C $\sim \frac{1}{2} \cdot N$ D $\sim \frac{2}{3} \cdot N$

(e) (*1 pt.*) Assume $N \geq 3$. How many stars are printed? (Choose the smallest correct estimate.)

```python
# python
i = 1
while N > 3:
    i = i + 1
    N = N // 3
    print("*")
for j in range(i + 1):
   for k in range(j + 1):
      print("**")
```

```java
// java
int i = 1;
while (N > 3) {
    i = i + 1;
    N = N / 3;
    StdOut.print("*");
}
for (int j = 0; j <= i; j++) {
    for (int k = 0; k <= j; k++) {
        StdOut.print("**");
    }
}
```

A $O(\log N)$ B $O((\log N)^2)$ C $O(N)$ D $O(N^2)$

(f) (*1 pt.*) Assume `random.shuffle` and `StdRandom.shuffle` shuffle the given array in-place uniformly at random. Further assume $N \geq 1$. How many stars are printed in expectation?

```python
# python
A = []
for i in range(N):
    A.append(i)
random.shuffle(A)
for a in A:
    print("*")
    if a == 0:
        break
```

```java
// java
int[] A = new int[N];
for (int i = 0; i < N; i++) {
    A[i] = i;
}
StdRandom.shuffle(A);
for (int a : A) {
    StdOut.print("*");
    if (a == 0) {
      break;
    }
}
```

A 1 B $O(\log N)$ C $O(\sqrt{N})$ D $O(N)$

(g) (*1 pt.*) Assume $N \geq 2$. How many stars are printed by the following recursive function?

```python
# python
def r(N):
    for i in range(N):
        print("***")
    if N > 2:
        r(N // 2)
        print("*")
        r(N // 2)
```

```java
// java
static void r(int N) {
    for (int i = 0; i < N; i++) {
        StdOut.print("***");
    }
    if (N > 2) {
        r(N / 2);
        StdOut.print("*");
        r(N / 2);
    }
}
```

A $O(\log N)$ B $O(N)$ C $O(N \log N)$ D $O(N^2)$

(h) (*1 pt.*) Assume we have a data structure `D` representing a set of integers, with an operation `insert`. Initially, the data structure is empty. Consider the following piece of code, where `A` is a given list of N integers:

```python
# python
d = D()
for i in A:
    d.insert(i)
```

```java
// java
D d = new D();
for (int i : A) {
    d.insert(i);
}
```

Suppose that the worst-case running time of this code is quadratic. What do you now know about the amortised running time of the operation `insert`?

- A It is impossible to infer an upper bound on the amortised running time from the given information.
- B `insert` runs in amortised constant time.
- C `insert` runs in amortised linear time.
- D `insert` runs in amortised quadratic time.

```python
class Mystery:
    def __init__(self):
        self.data = [0] * 4
        self.count = 0
        self.a = 0
        self.b = 0

    def is_empty(self):
        return self.count == 0

    def size(self):
        return self.count

    def resize(self, new_size):
        temp = [0] * new_size
        for i in range(self.count):
            temp[i] = self.data[(self.a + i) % len(self.data)]
        self.data = temp
        self.a = 0
        self.b = self.count

    def add(self, item):
        if self.count == len(self.data):
            self.resize(2 * len(self.data))
        self.data[self.b] = item
        self.b += 1
        if self.b == len(self.data):
            self.b = 0
        self.count += 1

    def remove(self):
        if self.is_empty():
            raise RuntimeError()
        item = self.data[self.a]
        self.data[self.a] = 0
        self.count -= 1
        self.a += 1
        if self.a == len(self.data):
            self.a = 0
        if self.count > 0 and self.count == len(self.data) // 2:
            self.resize(len(self.data) // 2)
        return item
```

Figure 1: Class `Mystery`, python version.

```java
public class Mystery {
    private int[] data;
    private int count;
    private int a;
    private int b;

    public Mystery() {
        data = new int[4];
        count = 0;
        a = 0;
        b = 0;
    }

    public boolean isEmpty() {
        return count == 0;
    }

    public int size() {
        return count;
    }

    private void resize(int newSize) {
        int[] temp = new int[newSize];
        for (int i = 0; i < count; i++) {
            temp[i] = data[(a + i) % data.length];
        }
        data = temp;
        a = 0;
        b = count;
    }

    public void add(int item) {
        if (count == data.length) resize(2 * data.length);
        data[b++] = item;
        if (b == data.length) b = 0;
        count++;
    }

    public int remove() {
        if (isEmpty()) throw new RuntimeException();
        int item = data[a];
        data[a] = 0;
        count--;
        a++;
        if (a == data.length) a = 0;
        if (count > 0 && count == data.length / 2) resize(data.length / 2);
        return item;
    }
}
```

Figure 2: Class `Mystery`, java version.

2. Mystery Class. The next few questions all concern the class defined in Figures 1 and 2.

(a) (*1 pt.*) What is printed?

```
mystery = Mystery()
mystery.add(1)
mystery.add(2)
print(mystery.remove())
mystery.add(3)
mystery.add(4)
print(mystery.remove())
print(mystery.size())
```

```
Mystery mystery = new Mystery();
mystery.add(1);
mystery.add(2);
System.out.println(mystery.remove());
mystery.add(3);
mystery.add(4);
System.out.println(mystery.remove());
System.out.println(mystery.size());
```

(b) (*1 pt.*) Show what the internal array `data` looks like after the operations in the previous question. Include the values of `a`, `b`, `count`, and the array length. Draw the array with indices and show which elements are occupied.

(c) (*1 pt.*) What data type does `Mystery` implement?

A Stack. B Queue. C Priority queue. D Deque.

(d) (*1 pt.*) What is the worst-case running time of a single `add()` operation? Here, n denotes the number of elements already in the data structure.

A $O(1)$. B $O(\log n)$. C $O(n)$. D $O(n \log n)$.

(e) (*1 pt.*) Consider a sequence of operations starting with a new, empty, freshly initialised `Mystery` object: 10 `add` operations followed by 5 `remove` operations. What will be the final length of the internal array `data`?

A 4. B 8. C 16. D 32.

(f) (*1 pt.*) What happens if you call `remove()` on a new, empty, freshly initialised `Mystery` object?

A Returns 0. B Returns the last removed element. C Throws `RuntimeException`. D Does nothing and returns `null`.

(g) (*1 pt.*) Professor Eager wants to modify `Mystery` to support a `peekLast()` operation that returns the most recently added item without removing it. Suggest an implementation of the `peekLast()` operation.

```
def peek_last(self):
    if self.is_empty():
        raise RuntimeError()
    # What goes here?
```

```
public int peekLast() {
    if (isEmpty()) throw new RuntimeException();
    // What goes here?
}
```

(h) (*1 pt.*) Why is it difficult to implement an efficient `removeAt(int index)` method for this data structure?

A Removing from the middle requires shifting all subsequent elements.

B The index parameter might be out of bounds.

C Finding the element at the given index takes $O(n)$ time.

D The circular array structure makes indexing complex.

(i) (*1 pt.*) For every $n \geq 100$, describe a sequence of n `add()` and `remove()` operations (starting from a new, empty, freshly initialised data structure) that shows why the amortised cost of `add()` and `remove()` of the implementation in Figures 1 and 2 cannot be better than linear. (It suffices to give correct examples for all $n \geq 100$, no further explanation is needed.)

(j) (*1 pt.*) Edit exactly one line of the code so that the amortised cost of `remove()` in any sequence of `add()` and `remove()` operations becomes constant. Your edit must not change the behaviour of the data structure. Which line of code do you change and how do you change it? (It suffices to write the line number and the new line, no further explanation is needed.)

(k) (*1 pt.*) Explain why the amortised cost of `add()` is constant in any sequence of `add()` operations (in the absence of `remove()` operations).

3. Operation of common algorithms and data structures.

(a) (*1 pt.*) Consider the following sequence of operations on a union–find data structure `QuickUnionUF` (as in [SW 1.5]). The data structure has been initialized with $n = 6$ elements (0, 1, 2, 3, 4, 5), and no other operations have been executed yet. Recall that `union`(x, y) in QuickUnionUF attaches the tree associated with x as a subtree to the tree associated with y. This is also the tie-breaking behavior in WeightedQuickUnionUF.

```
union(5, 2); union(0, 1); find(4); union(2, 1); union(2, 3); find(5)
```

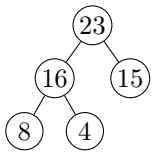
What is returned by the final `find(5)` operation?

A 0 B 1 C 2 D 3

(b) (*1 pt.*) Draw the resulting state of the `QuickUnionUF` data structure after the operations in (a).

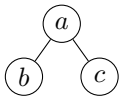
(c) (*1 pt.*) Now assume the operations in (a) are instead performed in a **`Weighted`**`QuickUnionUF` data structure (as in [SW 1.5]). Draw the resulting state.

(d) (*1 pt.*) After some operations in a max heap priority queue, my data structure looks like this:



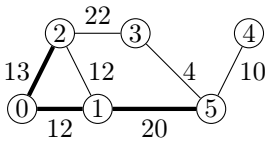
I now perform `insert(42)`. Draw the resulting state of the data structure.

(e) (*1 pt.*) Consider the following tree with three nodes:



Is it possible to choose a, b, c as three **different** integers such that this is the drawing of a maximum heap **and** a binary search tree? If yes, give an example. If no, briefly explain why not.

(f) (*1 pt.*) Here is a graph during an execution of Dijkstra's algorithm started at $s = 0$:



Edge weights are written next to the edges. The edges already added to the shortest path tree are drawn thick. Which will be the *next* edge that is added to the shortest path tree?

A 1–2 B 2–3 C 5–3 D 5–4

(g) (*1 pt.*) We're in the middle of an execution of insertion sort using alphabetic ordering, the underlined section indicates the already-sorted part of the array: `A D V E N T U R E`.[1] What is the situation after the next exchange?

(h) (*1 pt.*) Insert the 8 letters `B I O F U E L S` (in that order) into an initially empty 2–3 tree [SW 3.3] using alphabetic ordering. Draw the result.

(i) (*1 pt.*) Here is an array during the execution of quicksort (as in [SW 2.3]):

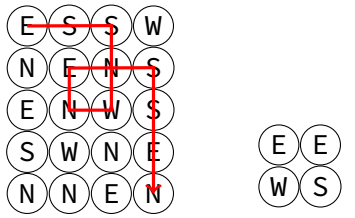
$[\mathbf{5}, 1, 4, 1, 3, 9, 2, 6, 5, 3]$

The first element is 5 and will be used as the next partitioning element (also called pivot). Draw the state of the array after the next partitioning.

[1] Recall the alphabetic order of the English alphabet: A B C D E F G H I J K L M N O P Q R S T U V W X Y Z.

Remark: In all questions below, use existing algorithms and data structures as much as you can, write clearly, and be brief. Faster is better. When stating running times, always choose the smallest correct estimate. Clear pseudocode or incredibly precise prose are preferred over syntactically correct Java or Python. If you use the same algorithm for some of the questions, just write that, like "see as 4c", instead of repeating yourself.

4. Design of algorithms Your parents have finally installed the $R \times C$ large grid of trampolines you've wanted for years. A single trampoline will bounce you in a predetermined compass direction: North ("Up"), South ("Down"), East ("Right"), or West ("Left"). On each bounce, you can choose an integer $s \geq 1$ that determines the strength with which you will jump: You will then bounce exactly s positions in the direction determined by the current trampoline. (Note that it is not possible to stay in place.) It's tea time, and you want to jump from the north–west corner to the south–east in as few jumps as possible, provided this is possible at all. Consider the examples below; the direction of each trampoline is indicated using the letters `NSEW`.



Sample 1 Sample 2

Sample 1 shows (as an arrow) a non-optimal sequence of 6 bounces corresponding to strength 2 for the first bounce, strength 2 for the second bounce, and strengths 1, 1, 2, 3 for the remaining bounces. In Sample 2, no solution is possible: the second jump would bounce you out of the garden.
To be concrete, let's agree that the input is given with positive integers R and C on the first line, followed by R lines of compass directions. Sample 1 would look like this:

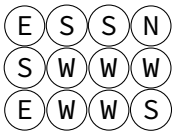
```
5 4
E S S W
N E N S
E N W S
S W N E
N N E N
```

The output is a sequence of non-negative integers that determines the strength of each bounce, or the word `impossible`. (In Sample 1, the output "`2 4 1`" would be correct.) If more than one correct optimal output is possible, any will do.

(a) *(1 pt.)* Model the problem as a graph problem: In particular, draw the graph corresponding to Sample 1. Make sure to include all edges, and indicate whether they are directed (and if so, in which direction) and/or weighted. In general, how many vertices and edges are there (expressed in terms of instance parameters such as R and C)? Use O or tilde notation if you find it convenient or relevant.

(b) *(1 pt.)* Which graph problem do you have to solve, how do you solve it, and what is the running time in terms of the instance parameters?

(c) *(1 pt.)* Let's add a twist: The strength required to bounce s positions is s, and you are tired. You want to minimise *strength consumption* instead of the number of bounces. For instance,



you can solve the problem in four bounces (output `2 2 2 3`), but that would consume $2+2+2+3 = 9$ strength. There is a better solution with five bounces that consumes 7 strength. Briefly discuss whether this twist changes the graph model, the best algorithm, or resulting running time.